

Practice 07: Reinforcement Learning and Neuroevolution

Intelligent Systems II

Mário Antunes

2026

Introduction

In this practice, we move beyond mimicking experts and enter the realm of **Reinforcement Learning (RL)** and **Neuroevolution**. Instead of being told what the “right” move is, our agents will learn through trial and error by interacting with an environment and receiving rewards.

We will explore: 1. **Tabular Q-Learning**: Using a Q-table to store state-action values. 2. **Turn-based RL**: Handling adversarial environments and state normalization. 3. **Deep Q-Learning (DQN)**: Using Neural Networks to approximate Q-values in continuous or large state spaces. 4. **Neuroevolution**: Optimizing Neural Network weights using Evolutionary Algorithms (Differential Evolution) instead of gradients.

Setup

Ensure you have the following packages installed:

```
pip install numpy keras jax jaxlib tqdm websockets pyBlindOpt
```

Set the Keras backend to JAX:

```
import os
os.environ["KERAS_BACKEND"] = "jax"
```

Part 1: Tabular Q-Learning - Hot/Cold

The simplest form of RL is **Tabular Q-Learning**. We maintain a table (or dictionary) where each entry $Q(s, a)$ represents the expected future reward for taking action a in state s .

Task:

1. **Environment:** Use the HotColdEnv where the state is the current range (low, high).
2. **Update Rule:** Implement the Q-Learning update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

3. **Exploration:** Use an ϵ -greedy policy to balance exploration (trying random guesses) and exploitation (using the best-known guess).

Hint:

- Initialize your Q-table as a dictionary where keys are (low, high) tuples and values are NumPy arrays of size 100.
 - A high penalty for “out of bounds” guesses (e.g., -10) helps the agent learn the game logic faster.
 - **Technical Hint:** Look at `support/06_reinforcement_learning/ex01_hot_cold/main.py` for the reference implementation.
-

Part 2: Adversarial RL - Tic-Tac-Toe

In turn-based games, the “next state” s' depends on the opponent’s move. Additionally, the agent needs to learn to play as both “X” and “O”.

Task:

1. **Normalization:** Normalize the board from the agent’s perspective. If the agent is “O” (-1), multiply the board by -1 so it always “thinks” it is player 1.
2. **Training:** Train your agent against different types of opponents:
 - **Random:** Good for initial exploration.
 - **Minimax:** Necessary to learn optimal play and defensive moves.
3. **Dual Learning:** Update the Q-values for *both* players during a single game simulation to double the training efficiency.

Hint:

- The reward for a win should be $+1$, a loss -1 , and a draw 0 .
 - Use a string representation of the board as the dictionary key for the Q-table.
 - **Technical Hint:** The `support/06_reinforcement_learning/ex02_tic_tac_toe/` folder contains a complete backend and a viewer. You can connect your agent to it via WebSockets.
-

Part 3: Deep Q-Learning (DQN) - Bouncing Ball

When the state space is continuous (like positions and velocities), a Q-table becomes impractical. We use a **Neural Network** to approximate the $Q(s, a)$ function.

Task:

1. **Experience Replay:** Store transitions $(s, a, r, s', done)$ in a buffer and sample random batches for training. This breaks the correlation between consecutive steps.
2. **Target Network:** Use a separate, slowly-updating network to calculate the target Q values. This stabilizes training.
3. **Expert Guidance:** Since the task (tracking a ball) is simple, use a heuristic “expert” to guide exploration in the early stages of training.

Hint:

- The input to your network is the state vector `[ball_x, ball_y, ball_vx, ball_vy, paddle_x]`.
 - The output is a 3-unit layer representing Q values for `[Left, Stay, Right]`.
 - **Technical Hint:** Check `support/06_reinforcement_learning/ex03_bouncing_ball`. The reward function is critical: give a small continuous reward for keeping the paddle under the ball. You can visualize the training progress using the provided backend and viewer.
-

Part 4: Neuroevolution - Snake

Neuroevolution doesn't use gradients. Instead, it treats the weights of a Neural Network as "genes" and uses evolutionary strategies to find the best set of weights.

Task:

1. **Perception Vector:** Instead of raw pixels, give the snake a concise vector:
 - Relative direction of food (4 values: Up, Down, Left, Right).
 - Immediate danger (4 values: Is there a wall or tail in the adjacent cells?).
2. **Objective Function:** The "fitness" of an agent is the total reward it collects in a game.
3. **Optimization:** Use **Differential Evolution** (via pyBlindOpt) to evolve a population of small MLPs.

Hint:

- Small networks (e.g., 8 inputs, 8 hidden neurons, 4 outputs) evolve much faster than large ones.
 - In the objective function, run multiple episodes per candidate and take the average to reduce noise from random food placement.
 - **Technical Hint:** Use the SnakeSimulator in support/06_reinforcement_learning/ for fast training, then connect to the WebSocket backend to see your snake in action.
-

Conclusion

Reinforcement Learning is powerful because it allows agents to discover strategies that their creators might not have considered. However, it is highly sensitive to **hyperparameters** (learning rate, discount factor, epsilon decay) and **reward design**.

Next Steps: * Compare Tabular Q-Learning vs. DQN on Tic-Tac-Toe. * Try changing the reward function in the Bouncing Ball game—does the agent become more "aggressive" or "cautious"? * Experiment with different evolutionary strategies (Genetic Algorithms vs. Differential Evolution) for the Snake agent.